

Немного извращений из мира прокси и VPN / Хабр

XTLS-Reality, ХНТТР, Naïverproxу и всякие там AnyTLS - это не интересно. Давайте копнем чуть глубже и посмотрим, где прячется настоящее безумие. Особенно учитывая, что мы живем во времена, когда даже самые, казалось бы, безумные вещи, могут оказаться весьма полезными чтобы не сойти с ума.

Pingtunnel

Идея классическая и простая как будерброд. ICMP пакеты могут нести полезную информацию. Полезную в смысле не “проверить, жив ли хост на той стороне”, а натурально какую-то стопку байт, которая, будут отправлена с клиента, будет получена сервером (если по пути не произойдет ничего плохого). А раз так, ничего не межает проксироваться через него. Самая известная реализация идеи - Pingtunnel. Сервер запускается и настраивается до смешного просто: отключили в sysctl системные ответы на пинги, чтобы не мешали, закинули бинарник на VPS, запустили его с ключом -server:

```
./pingtunnel -type server
```

И радуемся. Опционально можно добавить параметр -key с числом в диапазоне 0-2147483647 - это не полноценное шифрование (шифрование делает уже клиентский софт своим обычным TLS), а скорее простенькая защита от любопытных глаз, чтобы пакеты выглядели как случайный набор байт.

На клиенте - тот же самый бинарник, только с клиентскими параметрами:

```
pingtunnel -type client -l :4455 -s www.yourserver.com  
-sock5 1
```

(открывает socks-прокси на порту 4455),

А на Windows, Linux и - тадам! - Android можно использовать и красивый клиент [itismoej/pingtunnel-client](https://github.com/itismoej/pingtunnel-client)

Добавляем там новое подключение с URL типа `pingtunnel://your-server-ip?key=XXX&mode=vpn`, нажимаем Connect, и поехали.

Под Windows там есть баг в виде криво собранного бандла, что при попытке запустить подключение оно падает с "Unable to load asset" ошибкой, решается созданием папки `assets/binaries/pingtunnel/windows-amd64`, закидыванием в нее `pingtunnel.exe`, и выставлением переменной окружения `PINGTUNNEL_ASSETS_DIR` пути корня этой папки (там где лежит дира `assets`).

Ну и имейте в виду, что на Pingtunnel агрятся многие антивирусы, определяя его как HackTool, видимо потому что какие-то малвари использовали его в качестве транспорта.

Как оно работает? Ну, где как. При белых списках ICMP иногда не ходит вообще даже до белосписочных адресов, но кое-где ходит исправно почти куда угодно. У некоторых хостеров могут триггернуться фаерволлы, посчитав большой поток ICMP-пакетов с одного адреса DDoS-атакой. Но в остальных случаях - оно работает очень даже неплохо!

Важно: в Google Play и на гитхабе есть еще один PingTunnel клиент для Android от HexaSoftware (и гугл выдает его в первых строках по таким запросам). Его не советую - он работает как говно, иногда вообще не подключается, или работает очень медленно, что серфить невозможно. Лучше пользоваться тем клиентом, ссылка на который была выше.

Кстати, в XRay теперь есть аналогичный транспорт XICMP, правда реализация еще сырая, и документации, по традиции, почти никакой. А знаете где есть еще ICMP-туннель? В War Thunder в GoST, замечательном комбайне, про который рассказывалось в этой статье: GOST: швейцарский нож для туннелирования и обхода блокировок / Хабр

DNSTT, Slipstream, MasterDnsVpn

Тут идея другая - использование DNS для проксирования. Важно понять следующее: речь идет не про простое заворачивание ваших данных в пакеты DNS-протокола и отправки их на ваш сервер. Нет и нет. Тут именно что используются публичные DNS-сервера (например, вашего провайдера). Клиент делает DNS-запрос касательно домена `qJdESz0gPBiqojX1jK6kOLn1FAInEuF9enkIEWHIPpQ.yourdomain.com` (где в начале строки домена закодирована передаваемая информация). DNS-сервер вашего провайдера без понятия, что это за такой домен, поэтому он спрашивает `ns.yourdomain.com` (который тоже управляется вами) “Эй, брат, скажи - что это там у тебя за поддомен?”. А тот отвечает: у этого домена есть TXT-запись со значением `“aIWxjPTjL0LwKN-Q3gtd25fkVN73UwPk2XEupyG6Rgc”` (где тоже закодирована принимаемая информация), а DNS-сервер вашего провайдера это знание передает обратно клиенту. Вот так оно и гоняет данные-туда сюда, используя только провайдерский (или еще какой) DNS.

Естественно, при наивной реализации работает оно очень медленно и ненадежно, поэтому умные люди стали думать, как это оптимизировать.

Самым старым и известным вариантом является DNSTT, использующий алгоритмы из протокола KCP. На хабре есть замечательная статья DNSTT. DNS туннель для обхода блокировок / Хабр про его настройку. В XRay, судя по всему, очень похожая реализация недавно появилась под именем XDNS, но я не проверял - и времени не было, и DNSTT сам по себе уже прошлый век.

А теперь к более современным вариантам. Знакомьтесь: SlipStream. Почти то же самое, но использует механизмы протокола QUIC, и да - работает гораздо быстрее.

Оригинальная реализация: EndPositive/slipstream: High-performance multi-path covert channel over DNS - довольно глючная и забавная,

но есть еще переписанная на Rust Mygod/slipstream-rust: High-performance multi-path covert channel over DNS in Rust with vibe coding - и она работает очень хорошо.

Из мобильных клиентов я пробовал DNSTT.XYZ (не смотря на название, он поддерживает SlipStream) и SlipNet Lite (у него не перmissive лицензия, но очень хорош).

Есть и другие клиенты разной степени хорошеи, например плагин для Shadowsocks: Mygod/slipstream-plugin-android: SIP003 Android plugin using slipstream-rust

А теперь тяжелая артиллерия - MasterDnsVPN. Как говорит автор,

“По своей основной цели он похож на такие проекты, как DNSTT или SlipStream, но имеет принципиально иную структуру и подход к реализации. Эта система разработана с учетом совместимости со многими типами поведения резолверов и сложными сетевыми условиями, с целью сохранения максимально возможной стабильности и доставки данных даже в наихудших случаях.”

И у него действительно получилось. Среди всех мною попробованных протоколов, он самый быстрый и стабильный, а в сравнении с оригинальным DNSTT так просто небо и земля. Плюс огромное количество параметров, которые можно подкрутить для тонкой настройки.

Консольный клиент работает где угодно и на чем угодно, а еще есть два неплохих клиента для Android:

- Hidden-Node/MasterDnsVPN-AndroidClient: MasterDnsVPN Android client (community build) based on the upstream Go core—install signed APKs from Releases and connect to your own MasterDnsVPN server (domain/key/resolvers).
- RevocGG/MasterDnsVPN-AndroidGG: MasterDnsVPN Android Client

(второй мне нравится больше, первый как-то неаккуратно сделан).

Основной вопрос - какие DNS использовать. Некоторые публичные DNS могут оказаться недоступными в некоторых условиях. Некоторые DNS-сервера могут иметь ratelimit, в результате чего, например, через такой туннель через них еще можно будет как-то переписываться в Telegram и очень неспешно серфить, но вот видео уже не помотришь. А еще может быть, что провайдер перехватывает все нешифрованные запросы на 53 UDP-порт и переадресовывает их на свои сервера (но так делают далеко не все).

Поэтому стоит действовать так: сначала пробуем провайдерские DNS (не все клиенты умеют подхватывать их автоматически, стоит посмотреть в параметрах соединения и вбить вручную адреса), потом пробуем публичные типа 8.8.8.8 и 1.1.1.1 (вдруг доступны?), еще есть DNS-сервер MSK-IX 62.76.76.62, и верх цинизма - сервер НСДИ от Роскомнадзора 195.208.4.1 :)

Если ничего из вышеперечисленного не подходит, то идем куда-нибудь сюда: [DNS servers in Russian Federation](#), скачиваем список в виде текстового файла, заталкиваем в клиент, и пусть он проверяет. А чтобы не пришлось проверять слишком много, можно использовать простой Python-скрипт, который для начала отсекает точно недоступные варианты:

Скрипт тут

```
`#!/usr/bin/env python3
"""
Usage:
    ./resolve_check.py <ip_list_file> <domain> <output_file> [--
timeout 3] [--workers 50]
"""

import argparse
import ipaddress
import sys
from concurrent.futures import ThreadPoolExecutor, as_completed

import dns.resolver
import dns.exception

def load_ipv4_addresses(path: str) -> list[str]:
    """Read file, return unique IPv4 addresses preserving
order."""
    seen = set()
    ips = []
    with open(path, "r", encoding="utf-8") as f:
        for line_no, raw in enumerate(f, 1):
            line = raw.strip()
            if not line or line.startswith("#"):
                continue
            try:
                addr = ipaddress.ip_address(line)
            except ValueError:
                print(f"[skip] line {line_no}: not an IP ->
{line}", file=sys.stderr)
                continue
            if not isinstance(addr, ipaddress.IPv4Address):
                continue # ignore IPv6
            s = str(addr)
            if s not in seen:
                seen.add(s)
                ips.append(s)
    return ips
```

```

def can_resolve(ip: str, domain: str, timeout: float) -> bool:
    """Return True if `ip` (used as DNS server) successfully
    resolves `domain`."""
    resolver = dns.resolver.Resolver(configure=False)
    resolver.nameservers = [ip]
    resolver.timeout = timeout      # per-try
    resolver.lifetime = timeout     # total
    try:
        answer = resolver.resolve(domain, "A")
        return len(answer) > 0
    except (dns.resolver.NoAnswer,
            dns.resolver.NXDOMAIN,
            dns.resolver.NoNameservers,
            dns.exception.Timeout,
            dns.exception.DNSException,
            OSError):
        return False

def main() -> int:
    p = argparse.ArgumentParser(description="Test which IPv4
    addresses can resolve a domain.")
    p.add_argument("input", help="File with IP addresses, one per
    line")
    p.add_argument("domain", help="Domain name to resolve, e.g.
    example.com")
    p.add_argument("output", help="File to write working DNS
    server IPs to")
    p.add_argument("--timeout", type=float, default=3.0,
    help="DNS timeout in seconds (default 3)")
    p.add_argument("--workers", type=int, default=50,
    help="Parallel queries (default 50)")
    args = p.parse_args()

    ips = load_ipv4_addresses(args.input)
    if not ips:
        print("No IPv4 addresses found in input.",
        file=sys.stderr)
        return 1

    print(f"Testing {len(ips)} IPv4 address(es) against domain
    '{args.domain}'...")

    working: list[str] = []
    with ThreadPoolExecutor(max_workers=args.workers) as pool:
        futures = {pool.submit(can_resolve, ip, args.domain,
        args.timeout): ip for ip in ips}
        for fut in as_completed(futures):
            ip = futures[fut]
            ok = False
            try:
                ok = fut.result()
            except Exception as e:
                print(f"[error] {ip}: {e}", file=sys.stderr)
            status = "OK " if ok else "FAIL"

```

```

        print(f" {status} {ip}")
        if ok:
            working.append(ip)

    # Preserve original input order in the output
    order = {ip: i for i, ip in enumerate(ips)}
    working.sort(key=lambda x: order[x])

    with open(args.output, "w", encoding="utf-8") as f:
        f.write("\n".join(working) + ("\n" if working else ""))

    print(f"\nDone. {len(working)}/{len(ips)} resolved
    '{args.domain}'. Saved to {args.output}")
    return 0

if __name__ == "__main__":
    sys.exit(main())`

```

Bridge To Freedom

А вот тут уже идет тяжелая наркомания. Если вы устали от попыток “поймать” белый IP-адрес в Яндекс-Облаке и подобных, или для вас это слишком дорого, то наверняка задумывались об использовании других их сервисов для проксирования. Из самых дешевых вариантов у них есть *serverless functions*. Это такая функция на каком-нибудь языке программирования, которая не имеет состояния, и единственная задача которой - родиться, обработать один или несколько входящих запросов и сдохнуть. Из-за ограниченности времени жизни функций, и еще более важно - из-за того, что входящие запросы к таким функциям проходят через очень строгий яндексовский API Gateway, то использовать ХНТТР не получится. Зато оно все вполне официально поддерживает websockets.

“Ага, значит это можно использовать для проксирования”, - подумал автор уас-ws-bridge, оно же Bridge To Freedom.

Serverless-функция может реагировать на события “open”, “close”, “ondata”, и сама посылать данные в уже установленное соединение. Поэтому автор уас-ws-bridge попробовал сначала сделать “прозрачный” (не требующий каких-либо модификаций клиента) прокси, но работало оно

ужасно - через API Gateway пакеты с данными могли прилетать в неправильной последовательности, что ломало TLS и HTTP - Telegram еще как-то работал со скрипом, но серфить было очень тяжело с кучей обрывов.

Был еще вариант, где вместо оперирования TCP-потоками оно пересылало IP-пакеты (как в полноценном VPN), чтобы восстановление порядка данных и потерянных частей занимало TCP-стек операционной системы - оно работало, но катастрофически медленно.

В итоге самым стабильным и быстрым оказался вариант со своим собственным протоколом мультиплексирования и восстановления порядка пакетов, плюс если у клиента доступен не только endpoint serverless-функции, но и gRPC API самого API Gateway Яндекса, то функция используется вообще только для того, чтобы клиент и сервер “нашли” друг друга (узнали ID вебсокет-подключений друг друга), а дальше данные шлются между ними напрямую через API яндекса, и в итоге все работает гораздо быстрее.

В итоге идея простая: На VPS ставится серверная часть, на клиент - клиентская. Поскольку клиент и сервер не могут достучаться друг до друга напрямую (IP сервера не в белых списках), они оба подключаются по вебсокетам к облаку, где serverless-функция помогает им обмениваться ID вебсокетных подключений от облачного Gateway API, после чего они могут слать пакеты друг другу через этот самый Gateway API. А если у клиента недоступен адрес gRPC API самого Gateway API, то serverless функция ещё используется для того чтобы слать данные от клиента к VPS.

Позавчера (15.05.26) появилось вот такое обновление:

Эта версия серьезно оптимизирована. Хелпер теперь переупорядочивает фреймы по SeqID на приеме, регистрирует stream до отправки OPEN (так что первые DATA-

пакеты после OPEN_OK больше не теряются), использует асинхронную очередь записи на каждый stream (медленный локальный клиент больше не блокирует все остальные streams), и корректно делает half-close по FIN (HTTP-ответы больше не обрываются). На практике: подключение устанавливается заметно быстрее, и туннель работает значительно стабильнее, особенно на мобильных устройствах.

Я попробовал, и оно действительно теперь работает достаточно быстро и стабильно - у меня получилось выжать 40 мегабит на прием и 5 на отдачу.

В README есть подробное объяснение работы, инструкция по настройке серверной части и заливки serverless-функции в Яндекс. Функцию рекомендуется обязательно обфусцировать перед заливкой и поменять все пути со стандартных на нестандартные, чтобы Яндекс вас сразу не забанил.

В той же репе есть графический клиент для этого дела. Поскольку задача BTF - проброс TCP-потока, то работает он в паре в SOCKS- или VLESS-прокси, например v2RayN или v2rayNG: в v2RayN/v2RayNG вы настраиваете SOCKS- или VLESS-подключение к серверу, но обращаетесь на адрес localhost, где слушает BTF, он через инфраструктуру Яндекса передает все к вашему серверу, где подключается тоже на localhost, на котором слушает XRay или Dante. И все работает.

Клиент написан на .NET MAUI, есть сборки под Windows и под Android. Можно собрать под iOS самостоятельно, но понадобится Apple Developer Account, а еще MAUI теперь поддерживает сборку под Linux с бэкендами Avalonia и GTK4, поэтому в теории можно собраться и под Linux (ну либо использовать консольный Go-клиент, который работает везде).

Аудио-видеозвонки в мессенджерах

Nuff said. Тут без подробностей, ограничусь только ссылками:

KillTheCensorship/Turnel (уже не поддерживается, но может быть полезным для изучения, как оно сделано)

nil2x/cheburnet: Обход белого списка с помощью ВК.

TheAirBlow/Turnable: VPN core for stealthy tunneling through TURN or via SFU (пока что самый подвинутый вариант).

Да прибудет с вами сила.